

Modulo 1: Nivel de aplicacion. Tema 5: P2P

Arquitectura Peer-to-Peer (P2P)

- Cada cliente es tambien un servidor (nodos)
- Los nodos son todos iguales y se comunican directamente entre si
- Los nodos se conectan de forma intermitente, cambiando de IP (tiene una gestion compleja)
- Ejemplos: intercambio de ficheros (BitTorrent), criptomonedas (Bitcoin) VoIP (Skype)

Operaciones basicas

Registro

Proceso por el que un nodo se une a la red.

Existen dos soluciones basicas:

- Servidor de descubrimiento publico: sirve como "referencia" inicial, y proporciona las direcciones IP de otros nodos para poder empezar a conectarse a ellos.
 - Enfoque centralizado: punto de fallo
 - Mantenimiento costoso: no puede fallar o toda la operacion queda inoperativa
 - Un atacante puede aprender la topologia de la red preguntando a este servidor
- Lista de nodos "habituales" embebidos en el software
 - Necesaria una actualizacion muy frecuente

Busqueda de informacion (Lookup)

La informacion suele dividirse en trozos ("chunks") que son faciles de intercambiar y almacenar.

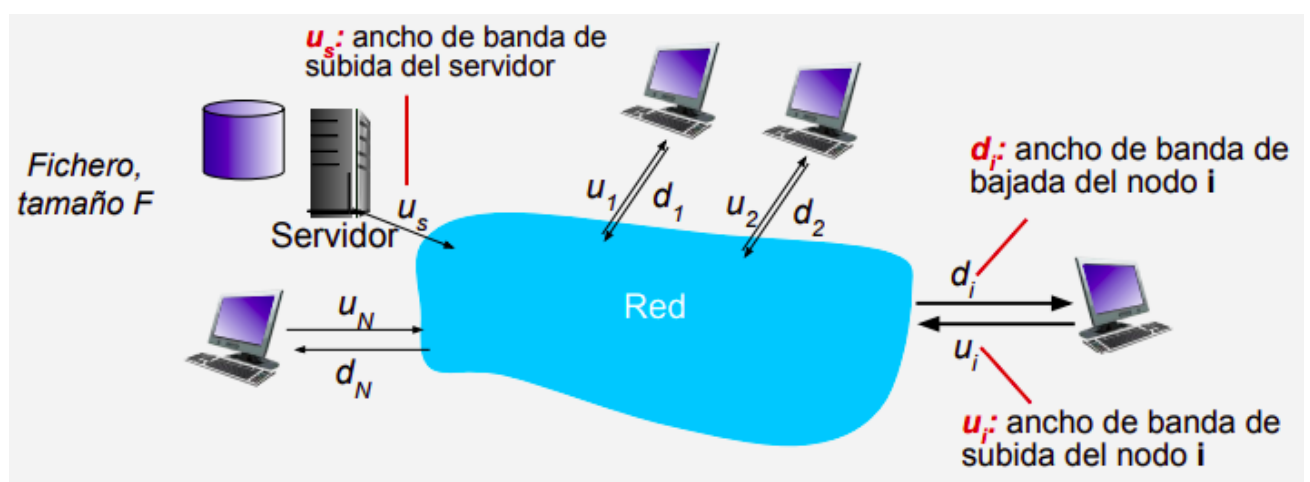
Esta operacion localiza en que nodo se encuentra cada porcion que necesita

Descarga de informacion (Retrieval)

Una vez localizada la informacion, el participante se conecta directamente a los nodos necesarios y comienza la descarga.

Puede (suele) descargar chunks de un mismo fichero de distintos nodos a la vez.

Distribucion de ficheros (C-S vs P2P)



(caso sobre el que se llevan a cabo los calculos posteriores)

Cliente-Servidor

- Servidor: suponemos que el servidor envia N copias del fichero de forma secuencial:
 - Tiempo para mandar una copia: $\frac{F}{u_s}$
 - Tiempo en mandar N copias: $\frac{NF}{u_s}$
- Cliente: cada cliente i tarda en descargar el fichero $\frac{F}{d_i}$
 - $\min(d_i)$ tiempo de descarga del cliente mas lento

$$D_{cs} = \max \left\{ \frac{NF}{u_s}, \frac{F}{\min(d_i)} \right\}$$

Peer-To-Peer

- Servidor: debe subir al menos una copia antes de que la distribucion P2P se pueda poner en marcha (luego los clientes se ayudan entre si): $\frac{F}{u_s}$
- Clientes: cada cliente debe descargar una copia: $\frac{F}{\min(d_i)}$
- Red: la red como un todo, debe descargar un total de NF a una tasa maxima $u_s + \sum u_i$, considerando que todos los nodos transmiten a la vez

$$D_{p2p} = \max \left\{ \frac{F}{u_s}, \frac{F}{\min(d_i)}, \frac{NF}{u_s + \sum u_i} \right\}$$

Conclusion

- En C-S, el servidor es el cuello de botella cuando N es grande.
- En P2P, la carga se reparte y es mucho más escalable. P2P es más eficiente cuando hay muchos nodos, porque el servidor no necesita manejar toda la carga por sí solo.

Clasificacion

Las redes P2P se pueden clasificar en base a las tecnicas de busqueda de informacion:

- Centralizada: existe algun tipo de servidor (servidor de descubrimiento), o super-nodos (como los groups-leaders de KaaZaa) que ayudan en la localizacion de la informacion
- Descentralizada: no hay topologia ni estructura especifica, y la peticion de informacion debe llegar a todos los nodos (flooding)

Arquitectura centralizada - Napster

Napster primer ejemplo de red P2P centralizada

- Diseñada para la comparticion de archivos musicales (MP3)
- Debido a su diseño centralizado, tenia informacion sobre los archivos que se compartian, por lo que fue denunciada por las compañías discograficas
- En 2002 fue obligada a cerrar
- Actualmente sigue existiendo como plataforma de pago, al estilo de Spotify

Busqueda de un archivo en arquitectura centralizada

1. El nodo que quiere buscar el fichero se conecta al servidor (centralizado) de descubrimiento y registra su contenido
2. El nodo que quiere buscar el fichero envia una peticion buscando el fichero
3. El servidor de descubrimiento le proporciona la IP de un nodo que contiene ese fichero
4. El nodo que quiere buscar el fichero se conecta directamente con el nodo que contiene el fichero y descarga el contenido

Características

- Transferencia de archivos **descentralizada** (ya que te conectas a un nodo cualquiera)
- Búsqueda de información **centralizada** (ya que te conectas al servidor de descubrimiento el cual se conoce)

Desventajas

- Punto único de fallo (si el servidor de descubrimiento falla, no se pueden conectar nodos)
- Rendimiento limitado (Cada búsqueda de archivos, conexión entre pares, actualizaciones, etc., pasa por el servidor)
- Problemas de copyright en el caso de compartir contenido ilegal

Arquitectura descentralizada

En función de la topología de la red, las redes P2P descentralizadas se pueden clasificar en:

- **Estructuradas:** organizadas de acuerdo a una topología específica, con un protocolo que asegura una búsqueda eficiente de datos, incluso si el recurso es extremadamente raro (poco compartido)
- **No estructuradas:** no imponen una estructura en particular, sino que se auto-organiza conforme los nodos se conectan a la red y establecen conexiones aleatorias entre ellos

Descentralizada - No estructuradas

Debido a su estructura, tienen algunas **ventajas**:

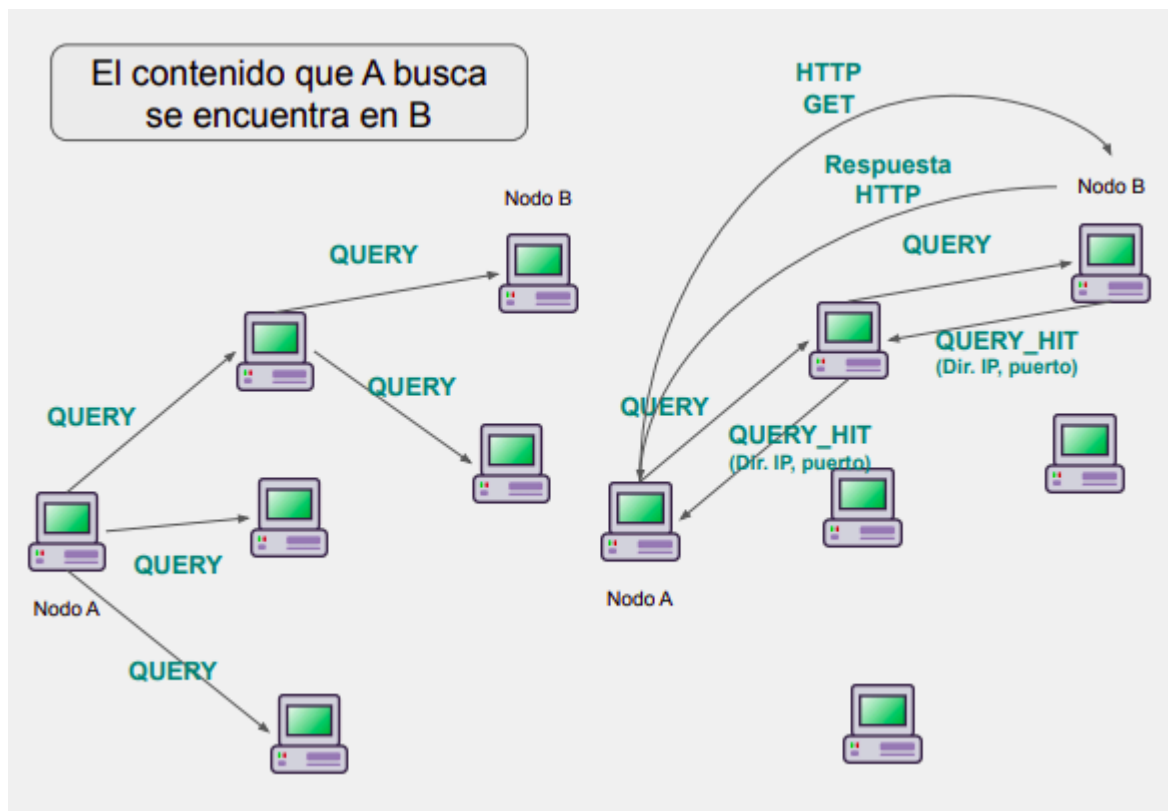
- Son fáciles de "construir", puesto que se auto-organizan. No es necesario un protocolo que ordene los nodos
- Como todos los nodos tienen el mismo papel (no hay super-nodos ni servidores de descubrimiento), estas redes son extremadamente robustas. Sin embargo:
- Esta (falta de) estructura **limita el rendimiento** de la búsqueda de información, que debe hacerse por inundamiento (flooding)

Cuando un nodo quiere encontrar un dato en la red, la consulta de búsqueda debe ser propagada a través de la red para encontrar tantos pares como sea posible que compartan los datos (elevado tráfico de señalización, uso de CPU). Además, al no existir correlación entre un nodo y el contenido gestionado por él, no hay garantía de que encuentre un par que contenga los datos deseados.

No estructurada - Gnutella

Una de las primeras redes en no depender de un servidor central:

- Cada nodo se conecta a un número limitado de nodos que utilizan el protocolo Gnutella también
- Existen dos tipos básicos de mensajes:
 - **Peticion** (query): enviado por un nodo a sus nodos directamente conectados con la identificación de un objeto que buscan
 - **Respuesta** (query response): si un nodo que recibe una petición tiene el contenido solicitado, responde indicando su dirección IP y puerto de conexión



Ejemplo de búsqueda de archivo:

1. Los pares envían mensajes QUERY a sus vecinos directos de la red
2. Los receptores reenvían el mensaje de consulta a todos sus vecinos, que a su vez lo reenvían a todos sus vecinos, etc.
3. Cuando un nodo recibe un mensaje QUERY, comprueba si la palabra clave coincide con alguno de los archivos que tiene a su disposición para compartir
4. Una vez encontrada una coincidencia, devuelve un mensaje QUERY_HIT, que contiene el nombre y el tamaño del archivo coincidente, junto con su dirección IP y puerto
5. El mensaje QUERY_HIT sigue la ruta inversa a la del mensaje QUERY, por lo que utiliza las conexiones TCP preexistentes. Si se reciben varios mensajes de respuesta, el usuario decide que archivo quiere descargar
6. El nodo inicial establece entonces una conexión TCP directa con el usuario deseado y envía un mensaje HTTP GET que incluye el nombre específico del archivo:
 - A. El archivo se envía con un mensaje de respuesta HTTP
 - B. Una vez recibido el archivo completo, se finaliza la conexión TCP directa

Estructurada

La red se organiza en una topología específica, normalmente utilizando variaciones del esquema de tablas hash distribuidas (DHT), que clasifican y organizan a los nodos utilizando pares clave valor. Así se consigue un alto grado de descentralización y, simultáneamente, eficacia y garantía de resultados.

1. Funciones hash

Una función hash es un algoritmo que transforma datos de entrada de tamaño variable en una salida de longitud fija, conocida como valor hash o resumen del mensaje. Este proceso es determinista, lo que significa que la misma entrada siempre producirá el mismo hash. Las funciones hash se utilizan ampliamente en informática y criptografía para diversas aplicaciones, como la verificación de integridad de datos y la autenticación.

Familia	Función	Longitud salida	Estado	Fecha publicación
MD	MD4	128	Insegura	1991
	MD5	128	Insegura	1992
SHA-1	SHA1	160	Insegura	1995
SHA-2	SHA256	256	Segura	2001
	SHA512	512	Segura	2001
SHA-3	SHA3-256	256	Segura	2015
	SHA3-512	512	Segura	2015

2. DHT (Distributed Hash Table)

- Organizan tanto los nodos (direcciones IP) como las claves (identificadores de contenido), para vincular unos con otros asignando un identificador en el rango $[0, 2^n - 1]$
- El identificador se calcula utilizando una función hash:
 - $\text{node_id} = \text{SHA256}(\text{Dir. IP}, i)$
 - $\text{key_id} = \text{SHA256}(\text{Nombre clave})$
- Ejemplo: Clave ("Guns 'N Roses")
 - $\text{key_id} = \text{SHA256}(\text{"Guns 'N Roses"}) = 4ff5dbbbc250c2ba7eed50b90b2ef8c3e9b27b3a789d5e25a286d2d69c843218$ **Protocolo Chord**

Los nodos se distribuyen en un anillo ordenado en función de su node_id . El contenido se almacena en el sucesor del nodo cuyo node_id es igual o mayor a key_id . *Ejemplo:* la clave K10 se almacenará en el nodo sucesor con ID igual o inmediatamente superior a 10 (N14).

Busqueda de Informacion

El nodo N8 busca una información con clave K54:

- Propaga su petición hasta que el nodo que contiene la información la recibe
- Se conecta directamente a él y la descarga. Aunque garantiza encontrar el contenido, esta solución tiene un rendimiento muy pobre:
- En el caso peor, todos los nodos (N) de la red deben transmitir un mensaje. En promedio, se transmiten $N/2$ mensajes.

Fingers (Busqueda de Informacion escalable)

Para mejorar la búsqueda, se utilizan una serie de "atajos" (denominados fingers) que reducen el número de nodos que deben ser consultados. Cada nodo tiene su propia tabla de atajos, de forma que el número total de mensajes se reduce. Las entradas para un nodo se calculan con un incremento exponencial:

$\text{finger}[i] = \text{sucesor}(\text{node_id} + 2^{**i}) \% n$

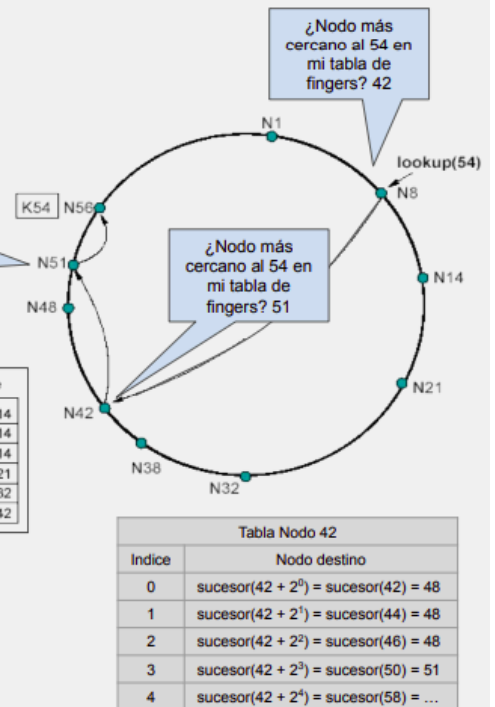
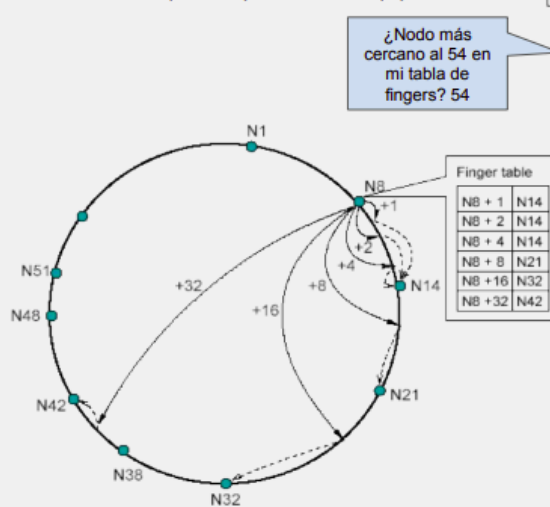
donde n es el número máximo de nodos en la red

Chord // Fingers

Tabla Nodo 8	
Indice	Nodo destino
0	$\text{sucesor}(8 + 2^0) = \text{sucesor}(8) = 14$
1	$\text{sucesor}(8 + 2^1) = \text{sucesor}(10) = 14$
2	$\text{sucesor}(8 + 2^2) = \text{sucesor}(12) = 14$
3	$\text{sucesor}(8 + 2^3) = \text{sucesor}(16) = 21$
4	$\text{sucesor}(8 + 2^4) = \text{sucesor}(24) = 32$
5	$\text{sucesor}(8 + 2^5) = \text{sucesor}(40) = 42$

Entrada 0

$$\text{sucesor}(8 + 2^0) = \text{sucesor}(8) = 14$$



Gestion de nodos

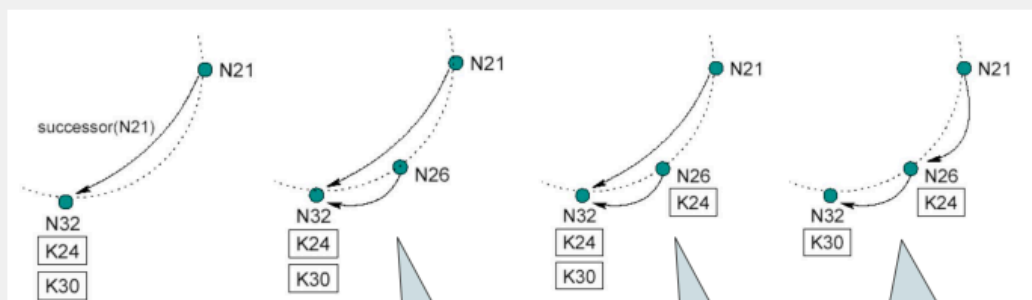
Cuando un nuevo nodo se une al sistema, los punteros de algunos nodos tendran que cambiar, o las busquedas no funcionaran correctamente Chord utiliza un protocolo de estabilizacion que corre periodicamente en background para actualizar estos puntos y la tabla de fingers

// Verifica el sucesor inmediato de n, y le notifica
n.stabilize()

```
x = sucesor.predecessor;
if (x in (n; sucesor))
    sucesor = x;
sucesor.notify(n);
```

x se establece al predecesor del sucesor de n, que sera n excepto cuando un nuevo nodo se ha unido recientemente entre x y su sucesor. En este caso, n establece su sucesor a x, y notifica a este de su propia existencia

Chord // Unión de un nuevo nodo



Situación inicial de la red

N26 se une a la red. Establece su sucesor como N32 y notifica a éste, que pone a N26 como su predecesor

K24 debe estar en N26, luego ésta la copia

La próxima vez que se ejecute **stabilize()**, detecta a N26 como su sucesor: N21 lo establece así y notifica a N26 para que lo anote como su predecesor

